

4장 공간단위 예측, 단위·피쳐·검증·해석

이 장에 나오는 Moran's I, 검증 점수, SHAP 값은 모두 `lecture_practice/chapter4/` 의 코드를 실제로 실행해 얻은 값

계산 구조를 설명하는 숫자에는 "예:"를 표시해 실제 실행값과 구분함. 실행하지 않은 결과는 제시하지 않음

1. 이 장의 핵심 질문

□ 이 장은 세 가지 질문에 답함

- **질문 1** — 격자 한 칸, 행정동 하나를 표의 한 행으로 만들려면 무엇부터 정해야 하고, "옆 동네 값" 같은 공간 피쳐는 어떻게 열이 되는가?
- **질문 2** — 가까운 곳끼리 값이 닮는 자료에서 무작위 검증은 왜 성능을 실제보다 높게 평가하는가? 이를 확인하는 방법은 무엇인가?
- **질문 3** — 예측 점수와 순위표가 나왔을 때, 그것을 어디까지 믿고 어떻게 읽어야 하는가?
- 이 장이 다루는 분석은 "어느 지역이 취약한가", "어느 지역에 시설이 부족한가"처럼 **공간단위마다 점수를 매기고 줄 세우는 예측**임
- 이 구조는 8장 이후의 정책 장과 14장 이후의 비즈니스 장에서 계속 다시 나옴. 여기서 절차를 한 번 제대로 세우면 뒤의 장들은 같은 틀의 반복임

2. 같은 모델, 세 개의 점수

○ 이 장의 실습 자료(포인트 500개)에 같은 모델을 적합하고, **검증 방법만 바꿔** 성능을 재면 이렇게 나옴

검증 방법	평균 R^2
무작위로 나눠 검증	0.869
공간 블록으로 나눠 검증	0.798
공간 군집으로 나눠 검증	0.349

□ 자료와 모델을 그대로 두고 검증 방법만 바꾸자 평균 R^2 가 0.869에서 0.349로 감소함

- 어느 점수가 적절한지는 **평가하려는 예측 상황에** 달려 있음. 이미 관측한 지역 사이를 예측한다면 무작위 검증이, 학습에 쓰지 않은 지역을 예측한다면 공간 분리 검증이 그 상황에 가까움
- 새 지역의 성능을 평가해야 하는데 무작위 검증 점수를 쓰면 성능을 실제보다 높게 평가할 수 있음
- Ploton 외(2020)는 아프리카 열대림 바이오매스 지도에서 무작위 교차검증과 공간 교차검증의 성능 차이가 크다고 보고했음. 이 결과는 공간 분리 없이 평가하면 새 지역 예측 성능을 제대로 알기 어렵다는 점을 보여 줌
- 이 장에서는 점수 차이가 생긴 이유(6절), 예측 상황에 맞는 검증 방법(7절), 결과 해석 방법(8절)을 차례로 다룸

3. 공간단위 예측의 여덟 단계

□ 공간단위 예측은 여덟 단계로 진행함. 앞 단계의 결과를 다음 단계에서 사용하므로, 순서를 지켜야 판단 근거를 확인할 수 있음

○ 검증 방법(6단계)을 정하기 전에 모델 설정(5단계)을 고르면, 검증 점수를 실제보다 높게 평가한 상태에서 모델을 선택할 수 있음

[1] 분석 단위·집계 규칙 정의	→ 단위 결정서, 집계 가중치
↓	
[2] 관측자료의 표 변환	→ 피쳐 행렬 x , 타깃 y
↓	
[3] 공간 피쳐 설계	→ 공간가중행렬 W , 공간 시차, 가용성 점검표
↓	
[4] 공간 구조 진단	→ 타깃의 Moran's I , 순열 검정 결과
↓	
[5] 모델 선택·적합	→ 기준모형 + 후보모형
↓	
[6] 공간 교차검증	→ 성능표, 폴드 간 표준편차
↓	
[7] 결과 해석	→ 피쳐 기여도, 잔차 진단
↓	
[8] 산출표·순위표	→ 단위별 예측표, 집계 순위표

그림 4.1 공간단위 예측의 여덟 단계와 단계별 산출물

○ 각 단계가 이 장의 어느 절에서 다루지는지는 다음과 같음

단계	무엇을 하는가	이 장의 절
1 분석 단위	표의 한 행을 무엇으로 돌지 정함	4절
2~3 표 변환·피쳐	관측치×피쳐 행렬을 만들고 공간 피쳐를 더함	5절
4 공간 진단	이웃한 지역의 타깃 값이 비슷한지 확인함	6절
6 공간 검증	검증을 어떻게 나눌지 정함	7절
5, 7~8 모델·해석·산출물	모델을 학습하고 결과를 읽음	8절

□ 먼저 패키지를 설치함. GPU는 필요 없고 노트북에서 끝까지 돌아감

```
cd geoai_public # 저장소 루트 - 이 장의 명령은 모두 여기서 실행함
pip install -r lecture_practice/requirements-student.txt
```

○ 4장이 쓰는 패키지만 따로 보려면 `lecture_practice/chapter4/code/requirements.txt`에 적혀 있으나, 설치하는 위 공통 파일 하나로 끝남

□ 이 장에서 실행할 코드는 세 개이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절
1	<code>4-1-spatial-features.py</code>	건물에서 형태·거리·위상 피쳐 추출	5.1
2	<code>4-2-spatial-cv.py</code>	Moran's I 와 검증 방법 비교	6~7
3	<code>4-5-grid-vulnerability-shap.py</code>	격자 취약성 예측·SHAP·순위표	8

○ 세 코드는 미리 준비한 `lecture_practice/chapter4/data/` 자료를 읽음

○ 세 코드 모두 인터넷도 GPU도 쓰지 않음

○ 본문에 인용한 출력은 코드 실행시 생성되는 `lecture_practice/chapter4/results/` 의 실행 로그에서 그대로 가져온 것이며, 어느 파일에서 왔는지를 출력 블록 첫 줄에 적어 두었음

4. 표의 한 행 정하기: 분석 단위와 집계

4.1 다섯 가지 후보

□ 분석에 들어가는 자료는 결국 표 하나로 정리되고, 그 표의 한 줄이 무엇인지가 첫 결정임

○ 이 결정에 따라 어떤 자료를 모을 수 있는지, 어떤 피처를 만들 수 있는지, 결과를 누구에게 어떤 단위로 전달할지가 모두 달라짐

단위	쓰임	특징
포인트	사건·시설 위치	원자료, 집계 전 단계
격자	취약성·위험 비교	균일 단위, 행정경계와 무관
행정구역	예산·정책 집행	통계 공표·집행 단위
필지·건물	안전·에너지·과세	미시 단위
네트워크 권역	접근성·서비스	이동시간 기반 기능 단위

○ **포인트**는 화재 신고 지점, 병원 위치처럼 사건·시설이 발생한 자리 그 자체임. 대부분의 분석은 포인트를 어떤 면 단위로 모으는 데서 시작함

○ **격자**는 국토를 같은 크기의 정사각형(예: 500m × 500m)으로 자른 단위임. 모든 칸의 면적이 같아 지역 간 비교에서 면적 차이를 통제할 필요가 없고, 행정 경계가 바뀌어도 단위는 그대로임

○ **행정구역**은 통계가 공표되고 예산이 집행되는 단위임. 분석 결과가 정책으로 이어지려면 결국 이 단위로 번역됨

○ **필지·건물**은 자산 하나하나를 판정하는 문제(건물 안전 진단, 에너지 효율 평가)에 씀

○ **네트워크 권역**은 도보 15분 안에 닿는 범위처럼 도로망 위의 이동시간으로 정의되는 단위임

4.2 칸을 바꾸면 결과가 바뀐다 — MAUP

□ 같은 현상이라도 지도를 어떤 크기·모양의 칸으로 잘라 세느냐에 따라 다른 결과가 나옴. 이 현상을 **가변적 공간 단위 문제**(Modifiable Areal Unit Problem, MAUP)라 함

○ **크기의 문제**: 고령인구 비율을 500m 격자로 집계하면 고령자가 많이 사는 작은 마을을 구분할 수 있음. 같은 자료를 1km 격자로 합치면 주변 지역 값과 평균되어 그 마을의 높은 비율이 낮아지고, 취약 지역을 구분하기 어려워짐

○ **경계의 문제**: 같은 마을이 행정동 경계를 어디에 긋느냐에 따라 취약 동에 속하기도 하고 평균적인 동에 속하기도 함

□ MAUP는 공간 자료를 집계할 때 생기는 성질임. 여러 값을 한 단위의 값으로 합치면 그 단위 안에 있던 지역 차이를 집계 결과만으로 확인하기 어려움. 단위의 크기와 경계에 따라 결과도 달라짐

○ 분석 단위와 보고 단위를 나누고(4.3), 결과에 사용한 단위를 적어 집계에 따른 차이를 확인함

4.3 현상 단위와 집계 단위

□ 취약성은 격자에서 발생하지만 예산은 행정구역에 배정됨

○ 처음부터 행정구역 단위로 분석하면 취약한 격자와 안정된 격자의 값이 평균되어 지역 안의 차이를 구분하기 어려움. 격자 예측만 제시하면 예산을 어느 기관에 배정할지 정하기 어려움

□ 그래서 **현상 단위에서 예측하고 집계 단위로 집계**하는 두 단계를 둬. 취약성은 격자에서 계산하고, 예산을 논의할 때 읍면동으로 다시 모음

○ 집계는 가중평균임. (가중치 × 예측값)의 합을 가중치의 합으로 나눔

예: 어느 행정구역 안에 격자 3개가 있고, 취약성 예측값이 40, 60, 80이라 하자

- ○ 모든 격자를 동등하게 두면(가중치 1, 1, 1): $(40+60+80)/3 = 60$

- ○ 인구가 각각 100, 500, 100이면(가중치 100, 500, 100): $(40 \times 100 + 60 \times 500 + 80 \times 100)/700 \approx 58.6$

- ○ 같은 예측값인데 답이 다름. **가중치를 명시하지 않은 구역 평균은 무엇을 평균한 값인지 알 수 없음**

주의. 집계 단위를 바꾸면 순위가 달라질 수 있음. 500m 격자에서 상위였던 지역이 1km 격자에서는 주변 값과 평균되어 상위권에서 빠질 수 있음. 순위표 제목이나 각주에 집계 단위와 가중치를 함께 적어야 함

5. 공간 피쳐 만들기

□ 실습 데이터는 미리 준비되어 있으므로 첫 번째 코드에서 바로 불러옴

```
python lecture_practice/chapter4/code/4-1-spatial-features.py
```

```
# 데이터: lecture_practice/chapter4/data/  
건물 200개, 지하철역 3개, 공간 포인트 500개  
격자 400개(20×20)
```

○ 세 자료의 쓰임이 각각 다름 — 건물 200개는 5.1의 피쳐 추출에, 포인트 500개는 6~7절의 자기상관·검증에, 격자 400개는 8절의 취약성 예측에 씀

5.1 관측치와 피쳐

□ 이 장의 모델은 관측치가 행으로, 각 관측치를 설명하는 피쳐가 열로 놓인 표를 입력받음

○ **관측치**는 분석의 대상이고, **피쳐**는 그 대상을 설명하기 위해 측정하거나 계산한 변수임. 예측 대상(취약성 점수 등)은 별도의 타겟 열로 둬

○ 8절이 쓰는 격자 자료의 첫 네 칸이 그 예임(`data/grid_vulnerability.parquet`에서 읽음)

격자	고령인구율	1인가구율	기초수급율	병원거리(m)	모의 NDVI(정규식생지수)	취약성
0	0.25	0.30	0.07	1,251	0.271	35.7
1	0.24	0.27	0.07	1,313	0.432	44.6
2	0.28	0.36	0.10	1,508	0.344	60.9
3	0.22	0.35	0.08	710	0.291	53.2

□ 이 표의 열들이 어디서 왔는지가 곧 이 절의 내용임

- 고령인구율·1인가구율·기초수급율 — 통계 자료를 격자별 열로 추가한 값
- 병원거리 — 격자 중심에서 최근접 병원까지의 **거리 피쳐**
- NDVI — 식생의 녹색 정도를 나타내는 지수
- 여기에 이웃 격자의 값을 요약한 **공간 시차**(5.4)를 열로 더할 수 있음

□ 두 번째 코드를 실행함. 건물 200개를 행으로 놓고 열 일곱 개를 만드는 과정이며, 위 표의 "병원거리"와 같은 종류의 거리 피쳐가 어떻게 계산되는지를 보여 줌

```
python lecture_practice/chapter4/code/4-1-spatial-features.py
```

```
# 출력 출처: lecture_practice/chapter4/results/4-1-spatial-features.log
import geopandas as gpd
import numpy as np

# [1] 건물 필지를 불러오고 기하 유효성부터 확인함
gdf = gpd.read_file(BUILDINGS_PATH)
gdf["geometry"] = gdf.geometry.make_valid()
# 출력: 불러온 건물: 200개 (좌표계 EPSG:32652)
#       유효하지 않은 기하 객체: 0개

# [2] 형태 피쳐 - 면적·둘레와 그 둘로 만드는 조밀도·중형비
gdf["area_m2"] = gdf.geometry.area
gdf["perimeter_m"] = gdf.geometry.length
gdf["compactness"] = 4 * np.pi * gdf["area_m2"] / (gdf["perimeter_m"] ** 2)
# 출력: 면적 범위: 167.0 ~ 3393.9 m²
#       조밀도 범위: 0.423 ~ 0.785
#       중형비 범위: 1.00 ~ 3.87

# [3] 거리 피쳐 - 건물 중심에서 최근접 지하철역까지, 그리고 중심점까지
gdf["dist_to_station"] = gdf.geometry.apply(
    lambda g: nearest_station_dist(g, stations))
gdf["dist_to_center"] = gdf.geometry.centroid.distance(center)
# 출력: 지하철역 거리: 96 ~ 3415 m
#       중심 거리: 141 ~ 4064 m

# [4] 위상 피쳐 - 50m 버퍼 안에 걸리는 다른 건물의 수
gdf["neighbors_50m"] = neighbor_counts
# 출력: 50m 내 인접 건물 수: 0 ~ 2
#       평균 인접 건물 수: 0.1
#       총 7개 공간 피쳐 추출 완료
```

□ 화면에 나온 값에서 읽을 것

- 만든 열 일곱 개는 세 유형임 — 도형 자체에서 나오는 형태 피쳐(면적·둘레·조밀도·중형비), 다른 대상까지 재는 거리 피쳐(지하철역·중심점), 주변에 무엇이 몇 개 있는지 세는 위상 피쳐(인접 건물 수)
- **조밀도**는 건물의 면적과 둘레로 모양이 정사각형에 얼마나 가까운지 나타냄. 값이 클수록 정사각형에 가까움, 정사각형의 조밀도는 0.785 임. 실습 자료의 건물도 이 값을 넘지 않으므로 계산이 맞음
- **위상 피쳐**는 주변 건물과의 관계를 숫자로 나타냄. 여기서는 각 건물에서 50m 안에 있는 건물 수를 셈, 가까운 이웃은 평균 0.1채 임. 건물 200개가 6km×6km에 퍼져 있어 이웃이 드뭄. 건물이 모인 자료에서는 이 값으로 지역 차이를 더 잘 구분할 수 있음

5.2 존 통계: 영상을 표의 한 칸으로

□ 격자 경계는 벡터이고 위성영상은 픽셀 격자임. 두 자료를 연결하는 계산이 **존 통계**(zonal statistics)임

□ 세 단계로 진행함

1. 격자 경계와 영상의 좌표계를 일치시킴 (2장에서 배운 그 작업)
2. 경계 안에 들어오는 픽셀을 골라냄

3. 그 픽셀들의 통계량을 계산해 표의 한 칸에 적음

□ 어떤 통계량을 고르느냐가 피처의 의미를 정함. 평균은 구역의 전반적 수준을, 표준편차는 구역 내부가 균질한지를, 임계값 초과 면적 비율은 조건을 만족하는 부분이 얼마나 되는지를 잴

○ 실제 위성영상으로 실습하면 격자 안 픽셀들의 NDVI 평균을 계산해 열로 추가함. 위 표의 모의 NDVI는 그 계산 결과가 아니라 존 통계를 설명하기 위한 가상 입력값임

5.3 어느 칸이 이웃인가 — 공간가중행렬

□ **공간가중행렬**(spatial weights matrix)은 지도에서 정한 이웃 관계를 숫자로 나타낸 표임. 보통 기호 **W**를 씀

○ 행과 열에 같은 지역을 같은 순서로 놓고, 각 지역 쌍이 이웃인지 또는 얼마나 가깝게 연결되는지를 칸마다 기록함

○ 이웃 관계를 어떻게 정하느냐에 따라 주변 값의 계산 결과가 달라짐. W는 지리적 관계를 분석에 반영하는 도구이며, 두 지역 사이의 인과관계를 증명하지는 않음

□ 이웃은 자료와 질문에 맞는 기준으로 정함

기준	이웃으로 보는 지역	예
변 인접(rook)	경계선을 맞댄 지역	격자에서 상하좌우 칸
꼭짓점 인접(queen)	경계선이나 꼭짓점이 닿은 지역	격자에서 대각선 칸도 포함
거리 기준	정한 거리 안에 있는 지역	중심점 간 5km 이내
k-최근접 이웃	중심점에서 가까운 순서로 고른 k개 지역	가장 가까운 3개 지역

□ 세 지역 A, B, C에서 A와 B만 이웃이고 C는 이웃이 없다고 하자. 이웃이면 1, 아니면 0으로 기록하면 다음과 같음

지역	A	B	C
A	0	1	0
B	1	0	0
C	0	0	0

○ 행은 기준 지역, 열은 이웃 후보 지역임. A행 B열의 1은 A와 B가 이웃이라는 뜻임

○ 대각선의 0은 지역을 자기 자신의 이웃으로 세지 않는다는 뜻임. C행처럼 값이 모두 0이면 이웃이 없는 지역임

□ 이웃 수가 지역마다 다를 때는 **행 표준화**를 할 수 있음 — 각 행의 값을 그 행의 합으로 나눠 행의 합이 1이 되게 함

○ 이웃이 네 곳이면 각 이웃에 0.25, 두 곳이면 각 이웃에 0.5를 줌. 그러면 주변 값의 가중평균을 구할 수 있음

○ 이웃이 없는 C는 행의 합이 0이므로 표준화한 평균을 계산할 수 없음. 분석에서는 이 지역을 어떻게 다룰지 따로 정해야 함

```
from libpysal.weights import KNN

w = KNN.from_dataframe(gdf, k=8) # 각 지점의 최근접 8개를 이웃으로
w.transform = "r"                # 행 표준화
```

○ 위 두 줄은 6~7절에서 실행하는 4-2-spatial-cv.py가 이웃을 정하는 부분과 같음 — 포인트 500개에 k=8 이웃과 행 표준화를 적용한 결과가 6.2의 Moran's I임

5.4 공간 시차: 이웃 값의 가중평균

□ **공간 시차**(spatial lag)는 W가 정한 이웃 지역의 값을 모아 만든 변수임. 행 표준화를 하면 이 값은 주변 지역 값의 가중평균이 됨

구분	시간 시차	공간 시차
기준	이전 시점	W가 정한 이웃 지역
질문	지난달 값은 얼마였나?	이웃 지역의 값은 얼마인가?
예	우리 동네 지난달 집값	이번 달 이웃 동네 집값의 가중평균

□ 시간 시차는 같은 지역의 과거 값을 현재 자료의 열로 추가함. 예를 들어 월별 자료에서 1시차는 바로 전 달의 값임

○ 시간 순서를 이용해 과거 값과 현재 값의 관계를 살필 수 있지만, 과거 값이 현재 값을 일으켰다는 뜻은 아님

□ 공간 시차는 같은 시점에 W로 정한 주변 지역의 값을 현재 자료의 열로 추가함. 이웃의 범위와 가중치는 5.3에서 정한 W를 따름

○ 예를 들어 A동의 이웃이 B동과 C동이고 두 지역의 집값에 같은 가중치를 주면, A동의 공간 시차는 B동과 C동 집값의 평균임

○ 가중치가 서로 다르면 단순 평균이 아니라 가중평균을 계산함. 따라서 공간 시차가 뜻하는 값은 W의 설정에 따라 달라짐

□ 아래 숫자는 계산을 설명하기 위한 예임. A동의 이번 달 집값은 5억 원, 지난달 집값은 4.9억 원, 이웃 B·C동의 이번 달 평균 집값은 6.5억 원이라고 하자

지역	기준 시점	집값	시간 시차(지난달)	공간 시차(이웃 평균)
A동	이번 달	5억 원	4.9억 원	6.5억 원

○ 시간 시차는 A동의 지난달 값을, 공간 시차는 이웃 동네의 이번 달 값을 나타냄

○ 공간 시차는 주변 값의 요약임. 이웃 지역이 A동 집값에 영향을 주었다거나 두 지역이 서로 영향을 주고받았다는 사실을 그 값만으로 확인할 수는 없음

5.5 만든 열을 써도 되는가 — 두 가지 점검 사항

□ 피처를 만들 때는 두 가지를 확인함

확인할 점	뜻	예
예측할 때 그 값을 알 수 있는가?	나중에 생길 값을 미리 쓰면 안 됨	화재 발생 예측에 진압 인력 수를 넣지 않음
검증 지역이 실제 예측 지역과 비슷한가?	가까운 지역을 섞어 나누면 새 지역 성능을 높게 볼 수 있음	새 지역 예측에는 지역을 나눠 검증함

□ 첫 번째는 **목표값 누수**(target leakage) 문제임

○ 올해 화재가 날지 예측하면서 올해 투입된 진압 인력을 피처로 쓰면 안 됨. 화재가 난 뒤에야 알 수 있는 값이기 때문임

○ 이런 값이 들어가도 프로그램은 오류 없이 실행될 수 있음. 점수가 높아져도 실제 예측 성능이 좋아졌다고 볼 수 없음

○ 피처마다 "예측하는 시점에 이 값을 어디서 얻는가?"를 확인함. 그때 알 수 없는 값은 빼야 함

□ 두 번째는 좌표와 검증 방법의 문제임

- 좌표를 넣으면 모델이 지역별 패턴을 배울 수 있음. 기록하지 않은 지역 조건도 위치에 따라 비슷하게 나타날 수 있음
- 무작위 검증에서는 검증 지점과 가까운 지점이 학습 자료에 남을 수 있음. 모델이 주변 패턴을 이용하면 새 지역을 예측할 때보다 점수가 높게 나올 수 있음
- 새 지역을 예측하려면 지역을 통째로 나눠 검증함
- 8절에서 좌표 피쳐와 검증 방법에 따른 차이를 실제 결과로 확인함

6. 공간 자기상관 재가: Moran's I

□ 세 번째 코드를 실행함. 이 코드에서 가까운 지점끼리 값이 닮았는지 확인하고, 다음 절에서 검증 방법을 비교함

```
python lecture_practice/chapter4/code/4-2-spatial-cv.py
```

○ 출력의 1~3구간은 이 절의 공간 자기상관 결과임. 뒤의 구간은 7절에서 다룸

6.1 계산 구조

□ 가까운 곳끼리 비슷한 값이 모이는 현상을 **공간 자기상관**(spatial autocorrelation)이라 함. 예를 들어 높은 값끼리 이웃하고 낮은 값끼리 이웃하면 공간 자기상관이 양수임

○ 그 모양을 한 숫자로 요약하는 지표가 **Moran's I**임. 계산할 때 어느 지역을 이웃으로 볼지는 5.3의 W로 정함

값이 놓인 모양	이웃한 값의 관계	Moran's I 해석
높음-높음, 낮음-낮음이 모임	이웃끼리 비슷함	양수: 비슷한 값이 모임
높음-낮음이 번갈아 놓임	이웃끼리 다름	음수: 다른 값이 이웃함
일정한 모양 없이 섞임	이웃 값에 뚜렷한 패턴이 없음	기대값 근처: 뚜렷한 전역 패턴이 적음

□ 네 칸이 일렬로 놓여 있고 값이 10, 12, 4, 6이라고 하자. 좌우로 맞닿은 칸만 이웃으로 보면 다음과 같음

1. 평균은 8이고, 각 칸의 편차는 +2, +4, -4, -2임
2. 이웃한 편차의 곱을 더함: $2 \times [(+2)(+4) + (+4)(-4) + (-4)(-2)] = 2 \times (8 - 16 + 8) = 0$. 앞의 2는 각 이웃 쌍을 양방향(예: 1→2와 2→1)으로 세기 때문에 곱함
3. 편차 제곱의 합은 $4 + 16 + 16 + 4 = 40$ 임
4. $I = (n / \text{가중치합}) \times (\text{교차곱} / \text{편차제곱합}) = (4 / 6) \times (0 / 40) = 0$

○ 값이 비슷한 이웃 쌍은 양수로, 값이 다른 이웃 쌍은 음수로 계산됨. 이 예에서는 양수와 음수가 상쇄되어 전체 I가 0임

○ 쉽게 보면 Moran's I는 "이웃끼리 닮았나, 달랐나?"를 요약함. 값 하나만으로 어느 지역에서 모였는지는 알 수 없고, 원인이나 영향 관계도 증명하지 않음

○ 따라서 전역 I가 0이어도 지역별 무늬가 없다고 단정할 수 없음. 어느 곳에 값이 모였는지는 LISA로 확인함

□ 아래 구간은 검증 방법을 고를 때 쓰는 **간이 참고표**임

Moran's I	공간 자기상관	검증 설계 참고
+0.7 ~ +1.0	강함	새 지역 예측이면 공간 분리 검증 필수

Moran's I	공간 자기상관	검증 설계 참고
+0.3 ~ +0.7	중간	공간 분리 검증 권장, 무작위 검증과 함께 보고
-0.3 ~ +0.3	약함	무작위 검증 사용 가능

○ 이 구간은 모든 자료에 통하는 엄격한 경계값이 아님. W의 이웃 정의와 검정 결과를 함께 보고, 실제 예측 대상이 새 지역인지도 고려함

6.2 실행 결과와 검증 설계 판정

□ 방금 실행한 코드의 앞 세 구간을 읽음. 포인트 500개에서 각 점의 가까운 8개를 이웃으로 정하고, 행 표준화를 적용한 결과임

```
# 출처: lecture_practice/chapter4/results/4-2-spatial-cv.log [1~3 구간]
--- 1. 공간 포인트 관측 데이터 불러오기 ---
불러온 포인트: 500개 (좌표계 EPSG:32652)
타겟 범위: -7.2 ~ 72.8
타겟 평균: 30.8, 표준편차: 15.1

--- 2. Global Moran's I ---
Moran's I 값: 0.8660
기대값 E[I]: -0.0020
Z-score: 40.8371
p-value: 0.0010
해석: 강한 양의 공간 자기상관 → 공간 CV 필수

--- 3. Local Moran's I (LISA) ---
유의한 클러스터 분류 (p < 0.05):
HH (Hot Spot): 159개
LH (Low-High): 1개
LL (Cold Spot): 134개
HL (High-Low): 0개
Not Significant: 206개
```

□ 화면에 나온 값은 이렇게 읽음

- Moran's I는 **0.866**임. 기대값 -0.002보다 훨씬 크므로, 이 자료에서는 비슷한 값끼리 가까이 모이는 경향이 강함
- **0.866**은 실제 자료에서 계산한 값, **-0.002**는 값이 500개 지역에 무작위로 놓였다고 볼 때의 기준값임. 기대값은 $-1/(500-1) \approx -0.002$ 로 약간 음수이며, 관측값은 이 기준보다 약 0.868 높음
- p값은 **0.001**임. 위치를 무작위로 섞어 계산한 결과만으로 관측된 패턴을 설명하기 어려움

□ Moran's I는 전체 패턴을 한 숫자로 보여 줌. 어느 지점 주변에 높은 값이나 낮은 값이 모였는지는 지역별로 계산하는 LISA로 확인함

LISA 분류	뜻	실습 결과
HH	높은 값 주변에 높은 값이 있음	159개
LL	낮은 값 주변에 낮은 값이 있음	134개
LH	낮은 값 주변에 높은 값이 있음	1개
HL	높은 값 주변에 낮은 값이 있음	0개
유의하지 않음	뚜렷한 국지 패턴을 확인하지 못함	206개

○ 이 결과를 보면 높은 값끼리 모인 곳과 낮은 값끼리 모인 곳이 모두 있음. Moran's I 하나만 볼 때 놓칠 수 있는 지역별 차이를 LISA가 보여 줌

- 이 자료는 가까운 지점끼리 값이 비슷함. 그래서 7절에서 표본을 무작위로 나눈 검증과 지역을 나눠 검증한 결과를 비교함
- Moran's I에 모든 자료에 똑같이 적용할 경계값은 없음. 이웃을 어떻게 정했는지와 검정 결과를 함께 보고 검증 방법을 고름

주의. Moran's I는 이웃을 정한 방식에 따라 달라짐. 결과를 보고할 때는 이웃 기준과 순열 검정 횟수도 함께 적음.

7. 공간 교차검증

□ 6절에서 실행한 코드의 뒤쪽 구간을 읽음. 아직 실행하지 않았다면 여기서 실행함

```
python lecture_practice/chapter4/code/4-2-spatial-cv.py
```

```
# 출처: lecture_practice/chapter4/results/4-2-spatial-cv.log [5~7 구간]
```

```
--- 5. Random CV vs Block CV 비교 ---
```

```
Random K-Fold CV:
```

```
R² scores: [0.866, 0.876, 0.855, 0.870, 0.878]
```

```
평균 R²: 0.869 +/- 0.008
```

```
블록 수: 36개 (5km 격자)
```

```
Spatial Block CV:
```

```
R² scores: [0.878, 0.519, 0.828, 0.917, 0.849]
```

```
평균 R²: 0.798 +/- 0.143
```

```
--- 6. Cluster CV ---
```

```
Cluster CV:
```

```
R² scores: [-0.179, 0.539, 0.508, 0.371, 0.509]
```

```
평균 R²: 0.349 +/- 0.271
```

```
--- 7. CV 기법 비교 요약 ---
```

기법	평균 R²	표준편차	과대추정
Random CV	0.869	0.008	+0.071
Block CV	0.798	0.143	+0.000
Cluster CV	0.349	0.271	-0.449

- 2절 첫머리의 표가 이 출력에서 나온 것임. 점수 차이가 생긴 이유(7.1), 분할 방법(7.2), 폴드별 점수 해석(7.3)을 차례로 설명함

7.1 무작위 분할에서 점수가 실제보다 높게 나오는 이유

□ 자료를 무작위로 나누면 가까운 지점이 학습 자료와 검증 자료에 따로 들어갈 수 있음

- 가까운 지점의 값은 서로 비슷한 경향이 있음. 모델은 학습 자료의 이웃을 보고 검증 지점의 값을 쉽게 맞힐 수 있음
- 이때 점수는 **새 지역을 얼마나 잘 맞히는지보다, 이미 본 지역 주변의 빈 곳을 얼마나 잘 채우는지**를 보여줌
- 무작위 검증 점수를 새 지역의 성능으로 해석하면 점수를 실제보다 높게 볼 수 있음. 이를 **공간 분할 누수(spatial split leakage)**라 함
- 목표값 누수는 예측할 때 알 수 없는 열을 빼서 고침(5.5). 공간 분할 누수는 학습 자료와 검증 자료를 지역별로 나눠 고침

7.2 공간을 나누는 네 가지 설계

○ 해법의 공통 원리는 하나 — 표본이 아니라 **지역을 통째로** 폴드에 배정함

설계	분할 방법	평가하는 상황
무작위 교차검증	행을 무작위로 분할	공간 구조가 없는 표본
블록 교차검증	좌표를 격자 블록으로 잘라 블록 단위 분할	인접하지 않은 블록의 예측
버퍼 교차검증	검증지 주변 학습표본을 제거	가까운 이웃 없이 예측
군집 교차검증	좌표를 군집으로 묶어 군집 단위 분할	학습하지 않은 권역으로 이전

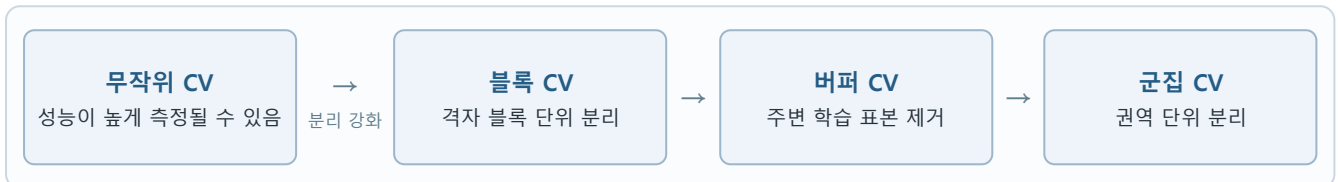


그림 4.2 검증 설계별 공간 분리 수준

- 오른쪽으로 갈수록 학습과 검증의 공간 분리가 강해지고, 재는 성능은 "학습하지 않은 지역"의 성능에 가까워짐
- 어느 설계를 쓰지는 모델이 실제로 놓일 상황에 맞춤(Roberts et al., 2017). 같은 도시 안의 빈 격자를 메우는 용도면 블록 검증이, 다른 시군으로 모델을 이전할 계획이면 군집 검증이 그 상황에 대응함
- 각 표본에 블록 식별자를 부여하고, 같은 블록이 같은 폴드에 들어가도록 `GroupKFold` 로 나눔

```

from sklearn.model_selection import GroupKFold, cross_val_score

# 좌표를 5km 격자로 잘라 블록 식별자 부여
gdf["block_id"] = ((gdf.geometry.x // 5000).astype(int) * 10000
                  + (gdf.geometry.y // 5000).astype(int))

# 같은 블록은 같은 폴드로 - 블록 단위 분리
scores = cross_val_score(model, X, y, scoring="r2",
                        cv=GroupKFold(n_splits=5), groups=gdf["block_id"])
  
```

- 위 코드가 첫머리 출력의 5구간을 만들었음 — 포인트 500개가 5km 블록 36개로 나뉘고, 같은 블록은 통째로 한 폴드에 들어감

7.3 실행 결과 읽기: 검증 방법 비교

- 성능지표는 회귀에서 R^2 를 씀 — "모든 단위에 평균을 예측하는 규칙"과 비교해 모델이 설명한 변동의 비율. $R^2=0$ 이면 평균 예측과 같은 수준이고, **음수이면 평균 예측보다 나쁨**
- 2절에서 본 표를 이제 전체 열로 봄. 첫머리 출력의 5~7 구간을 옮긴 것이며, 폴드 최저는 6구간의 R^2 목록에 찍힌 값임

검증 설계	평균 R^2	폴드 간 표준편차	폴드 최저
무작위 5-폴드	0.869	0.008	—
블록 CV	0.798	0.143	—
군집 CV	0.349	0.271	-0.179

□ 세 가지를 순서대로 읽음

1. **공간 분리를 늘리자 평균 R^2 가 감소함** — $0.869 \rightarrow 0.798 \rightarrow 0.349$. 무작위 검증 점수는 가까운 표본을 학습과 검증에 나누는 영향으로 높게 평가됐을 수 있음

2. **폴드 간 표준편차는 커짐** — 0.008 → 0.143 → 0.271. 군집 검증 점수는 제외한 권역에 따라 차이가 큼
3. **폴드 최저 -0.179** — 군집 검증의 어느 한 권역에 대해서는 모델이 평균 예측보다 못했다는 뜻. 평균 0.349는 이런 폴드를 포함한 값이므로, 확장을 계획한다면 평균이 아니라 최저 폴드를 근거로 삼음

8. 격자 취약성 예측과 결과 해석

□ 네 번째 코드를 실행함. 8절의 수치가 전부 이 한 번의 실행에서 나옴

```
python lecture_practice/chapter4/code/4-5-grid-vulnerability-shap.py
```

○ 출력은 다섯 부분임 — 자기상관 진단(8.1), 세 가지 검증(8.2), SHAP 기여도와 잔차 진단(8.3), 우선순위 표(8.4), 그리고 시드를 바꿔 반복한 검증 결과

8.1 자료와 기준모형

□ 자료는 지역을 20×20으로 자른 격자 400개임. 피쳐는 5.1의 표에 나온 다섯 열에 대중교통 거리와 좌표(x, y)를 더한 8개

○ 타깃(취약성)의 Moran's I는 **0.817**(p=0.001) — 6.2의 판정 표에서 공간 분리 검증이 필수인 구간

```
# 출처: lecture_practice/chapter4/results/4-5-grid-vulnerability-shap.log
분석 단위: 400개 격자(20×20), 피쳐 8개, 개인 식별 불가
```

```
[4.3 공간 자기상관] Global Moran's I = 0.817 (p = 0.001)
→ I>0.3, 유의 → 공간 자기상관 강함, 공간 CV 필수(아니면 누수)
```

○ **기준모형**(baseline model)은 같은 예측 문제를 푸는 간단한 비교 대상임. 결과를 본 뒤 고르지 않고, 분석 전에 정함. 회귀의 기본 기준은 학습 자료의 타깃 평균만 예측하는 모형이며, 이때 R²는 0임. 피쳐를 활용하는 간단한 기준이 필요하면 같은 피쳐를 쓰는 선형회귀를 추가할 수 있음

○ 이 실습 코드는 랜덤 포레스트의 공간 교차검증을 수행하며, 선형회귀는 계산하지 않음. 따라서 여기서 R²는 평균 예측 기준과 비교해 해석하고, 선형회귀와의 성능 비교는 하지 않음. 별도 기준모형을 비교할 때는 같은 자료 분할과 성능 지표를 적용해야 함

8.2 랜덤 포레스트

□ 이 실습의 모델은 **랜덤 포레스트**(Random Forest)임(Breiman, 2001). 결정 트리 하나는 학습자료가 조금만 바뀌어도 모양이 크게 달라지는데, 이 불안정을 여러 트리의 평균으로 줄이는 방법임

1. 원자료 n행에서 중복을 허용해 n행을 다시 뽑음 (트리마다 다른 표본)
2. 각 분기에서 전체 피쳐를 다 보지 않고 무작위로 고른 일부에서만 최적 분기를 찾음 (트리마다 다른 피쳐 후보)
3. 트리를 깊게 키움
4. 1~3을 수백 회 반복하고, 회귀는 예측의 평균·분류는 다수결로 결합함

○ 2단계가 핵심임. 표본만 다시 뽑으면 강한 피쳐 하나가 모든 트리의 최상단을 차지해 트리들이 서로 닮게 되고, 닮은 예측은 평균해도 분산이 줄지 않음. 피쳐까지 무작위로 골라야 트리들이 실제로 달라짐

○ 격자 자료에 적합한 뒤 세 가지 검증으로 잼. 코드는 비교를 위해 XGBoost도 같은 조건으로 함께 돌림

```
# 출처: lecture_practice/chapter4/results/4-5-grid-vulnerability-shap.log
[4.5 공간 CV 비교] Random vs Block vs Cluster (R²)
모델      Random   Block   Cluster   과대추정(R-B)
RF         0.639    0.535    0.373     +0.104
```

XGBoost 0.629 0.490 0.183 +0.139
→ Random CV가 Block/Cluster보다 높다 = 공간 누수로 인한 과대추정.

□ 화면에 나온 값에서 읽을 것

- 공간 분리를 늘리자 점수가 낮아짐. 무작위 검증 R^2 0.639 를 학습에 쓰지 않은 지역의 성능으로 보고하면 실제보다 높게 평가할 수 있음
- 무작위 CV에서는 RF(0.639)가 XGBoost(0.629)보다 높아 RF를 고름. 군집 CV에서도 RF(0.373)가 XGBoost(0.183)보다 높으므로, 이 표에서는 모델 순위가 바뀌지 않음. 다만 무작위 CV 점수는 두 모델 모두 군집 CV 점수보다 높아 새 권역 성능을 높게 평가함

8.3 SHAP: 예측을 피처별 기여로 분해

□ 모델이 어느 열을 보고 예측했는지 확인하는 도구가 **SHAP**임(Lundberg & Lee, 2017). 개별 예측 하나를 피처별 기여값으로 분해함

- 계산은 각 피처를 추가하기 전후의 예측값 차이를 비교함. 가능한 추가 순서에서 나온 차이를 평균해 피처별 기여값을 구함

□ 예: 평균 예측 45점, 고령인구율만 반영하면 52점, 병원 거리만 반영하면 48점, 둘 다 반영하면 59점이라 하자

- 고령인구율을 먼저 추가하면 기여는 $52-45 = 7$ 점 임. 병원 거리를 먼저 추가한 뒤 고령인구율을 추가하면 기여는 $59-48 = 11$ 점 임
- 두 순서를 평균: 고령인구율의 SHAP = $(7+11)/2 = 9$ 점. 같은 방식으로 병원 거리의 SHAP = 5점
- $45 + 9 + 5 = 59$ 로 예측값이 정확히 복원됨. SHAP 값은 예측값과 같은 단위(여기서는 취약성 점수)를 가짐

- 같은 실행이 짝은 피처별 평균 절대 SHAP과 잔차 진단

```
# 출처: lecture_practice/chapter4/results/4-5-grid-vulnerability-shap.log
[4.5 모델 해석] SHAP 피처 기여도 (RandomForest)
피처          평균|SHAP|
elderly        4.003
dist_hosp      2.476
y              2.123
basic          1.365
dist_transit   1.354
single_hh      1.035
x              0.795
ndvi           0.456

[누수 점검] 잔차 Moran's I = 0.663 (p = 0.001)
주의: 잔차에 공간 구조가 남음 → 공간 피처/모델 보강 여지.
```

□ 화면에 나온 값에서 읽을 것

- 열 이름은 차례로 고령인구율·병원 거리·좌표 y·기초수급율·대중교통 거리·1인가구율·좌표 x·모의 NDVI임
- **좌표 y가 3위**임 — 좌표 y가 예측에 크게 기여한 피처라는 뜻임. 모델이 위치와 함께 변하는 취약성 패턴을 좌표로 추정했을 수 있음. 이는 무작위 검증 R^2 (0.639)와 군집 검증 R^2 (0.373)의 차이와도 관련됨
- 예측 잔차(실제-예측)의 Moran's I는 **0.663**($p=0.001$)임. 예측 오차가 공간적으로 비슷한 값을 보임. 좌표를 사용해도 지역별 차이를 모두 설명하지 못했으므로 이 한계를 결과에 적음

주의. SHAP 값은 모델 예측에 각 피처가 기여한 정도를 나타내며 인과관계를 설명하지 않음. 병원 거리의 SHAP 값이 크더라도 병원을 지으면 취약성이 감소한다고 결론 내릴 수 없음. 모델이 병원 거리를 도시화 수준 같은 다른 조건의 대리 변수로 사용했을 수 있음

○ 의사결정에는 모델 점검과 추가 조사 대상 선별에 활용함. 평균 절대 SHAP은 모델이 전체 예측에서 어떤 피처를 많이 활용했는지 보여 줌. 개별 격자의 SHAP은 그 격자의 예측을 어떤 피처가 높이거나 낮췄는지 설명함. 이 정보를 보고 예측 근거가 타당한지 확인하고, 현장 조사나 자료 검증이 필요한 격자를 고를 수 있음

○ SHAP 순위만으로 지원 대상을 확정하거나 정책 효과를 예측하지 않음. 실제 지원 여부는 현장 확인, 지원 자격, 비용 등 별도 기준을 함께 검토해 결정함. 병원 신설처럼 어떤 조치가 취약성을 바꾸는지 알려면 SHAP이 아니라 인과 효과를 평가하는 분석이 필요함

8.4 순위표의 역할과 활용

□ 순위표는 모델이 예측한 취약성 점수가 높은 격자부터 정렬한 표임. 실제 취약성을 직접 측정한 값이나 지원 대상 확정표가 아님. 점수가 높은 지역을 먼저 살펴볼 후보로 찾는 데 씀

○ 실제 업무에서는 조사 인력이 제한될 때 현장 확인 순서를 정하거나, 자료 오류와 예측 근거를 검토할 후보를 추리는 데 활용할 수 있음. 순위만으로 지원을 결정하지 않고 현장 상황과 지원 기준을 확인함

○ 이 실습 코드는 400개 격자 전체로 모델을 학습한 뒤 같은 격자를 예측해 순위를 만들. 따라서 아래 표는 순위 작성 과정을 설명하는 예시이며, 새 지역의 실제 우선순위로 사용하면 안 됨. 실제 의사결정에는 학습에 쓰지 않은 지역에서 성능을 확인한 예측값이 필요함

○ 같은 실행이 출력한 예측 점수 상위 5개 격자

```
# 출처: lecture_practice/chapter4/results/4-5-grid-vulnerability-shap.log
[정책] 취약성 예측 상위 5개 격자(우선지원 후보)
cell elderly basic dist_hosp pred priority_rank
56 0.32 0.13 1575.86 59.40 1
16 0.30 0.06 1998.16 57.50 2
2 0.28 0.10 1507.67 56.66 3
35 0.26 0.10 1754.41 56.50 4
41 0.31 0.10 1693.43 54.96 5
전체 우선순위 저장 → ch4_vulnerability_priority.csv
```

○ 1위 점수는 59.40, 5위 점수는 54.96이며, 두 예측값의 차이는 4.44 점임. 400개 격자 전체의 순위는 results/ch4_vulnerability_priority.csv 에 저장됨

□ 순위표를 볼 때는 다음을 확인함

1. **순위 기준 확인** — 순위는 예측 점수가 높은 순서임. 모델 없이 고령인구율만으로 정한 순위와 비교하면, 모델이 추가로 반영한 조건이 무엇인지 찾을 수 있음
2. **비슷한 점수는 묶어 보기** — 점수 차이가 작으면 순서가 확실하다고 단정하지 않음. 개별 등수보다 우선 확인할 상위 후보 그룹으로 다룸
3. **선정 경계 확인** — 예를 들어 상위 20개를 조사한다면 20위와 21위 점수도 비교함. 두 점수가 비슷하면 순위 하나로 조사 여부를 가리지 않음
4. **예상 밖 순위 확인** — 고령인구율이 낮는데 높은 순위에 오른 격자는 다른 피처의 기여와 입력 자료를 확인함. 필요하다면 현장 조사로 실제 상황을 확인함

9. 핵심 요약

○ 검증 방법이 점수를 바꿈. 같은 자료·같은 모델에서 무작위 0.869, 블록 0.798, 군집 0.349. 어느 값을 쓸지는 재려는 능력 — 빈 곳 메우기인지 새 지역 예측인지 — 이 정함 (2절, 7절)

○ 여덟 단계는 정한 순서대로 진행함. 앞 단계의 결과를 다음 단계에서 사용함. 검증 방법을 정하기 전에 모델을 고르면 실제보다 높게 평가된 점수에 맞춰 설정할 수 있음 (3절, 그림 4.1)

- **첫 결정은 표의 한 행.** 포인트·격자·행정구역·필지·네트워크 권역 중 무엇을 한 행으로 두는지가 자료·피처·전달 단위를 정함. 집계 단위를 바꾸면 결과가 바뀌는 MAUP는 없앨 수 없으므로, 현상 단위에서 예측하고 집계 단위로 가중평균해 모으며 가중치를 명시함 (4절)
- **공간 피쳐는 이웃 정의에서 시작함.** 공간가중행렬 W로 이웃을 정하고 행 표준화한 뒤, 이웃 값의 가중평균인 공간 시차를 열로 만듦. 원변수와 시차를 비교해 주변보다 값이 높은지 낮은지 확인함 (5절)
- **누수는 두 종류이고 처방이 다름.** 목표값 누수(예측 시점에 없는 값이 열에 들어감)는 열을 빼서 고치고, 공간 분할 누수(인접 표본이 학습·검증에 나뉘어 들어감)는 나누는 방식을 바꿔 고침. 둘 다 오류 없이 통과하므로 사람이 확인해야 함 (5.5, 7.1)
- **Moran's I로 이웃한 값의 유사성을 확인함.** 실습 자료는 0.866($z=40.84$)으로 값이 비슷한 지점이 가까이 모임. 단, I가 0에 가까워도 지역별 패턴이 없다고 단정할 수 없음 — 4칸 예에서 I가 0이지만 이웃 값은 일부 비슷하고 일부 다름 (6절)
- **공간 교차검증은 지역을 통째로 나눔.** 블록→버퍼→군집 순으로 분리가 강해지고, 점수는 낮아지며 폴드 간 변동은 커짐. 재러는 차이가 폴드 간 표준편차보다 작으면($0.071 < 0.143$) 그 차이를 결과로 인용하지 않음 (7절, 그림 4.2)
- **결과 해석은 세 값을 함께 봄.** 좌표 y가 SHAP 3위면 예측에서 좌표 기여가 컸다는 뜻임. 잔차 Moran's I가 0.663이면 모델이 설명하지 못한 지역별 패턴이 남아 있음. 1~5위 차이(4.44점)가 오차 수준 안이면 개별 순위 대신 상위 그룹으로 보고함. SHAP은 인과를 설명하지 않음 (8절)

10. 과제

□ 이번 장의 과제는 **검증 점수 두 개의 차이를 직접 확인하고, 그 차이의 의미를 판단하는 것**

- 명령 한 줄이면 제출할 파일이 생성됨

```
python scripts/submit.py 4 --id 학번 --name 이름
```

□ 이 명령이 대신 처리하는 작업: 미리 제공한 자료로 대표 실습 `4-2-spatial-cv.py` 실행, 제출용 파일 생성

- 실행이 끝나면 `submissions/` 폴더에 `제출_4장_학번.md` 파일이 생기며, 파일에 실행 결과가 포함됨

○ **눈여겨볼 곳은 무작위 교차검증 점수와 공간 블록 교차검증 점수의 차이**

- 맨 아래 빈칸 세 개를 채우면 됨

1. 눈에 띈 숫자 하나를 그대로 옮겨 적고, 왜 눈에 띄었는지 한 문장으로
2. 그 숫자가 왜 그렇게 나왔는가
3. **무작위 교차검증 점수를 그대로 보고서에 실었다면, 그 보고서는 무엇을 잘못 말하게 되는가?**

□ **3번이 이 과제의 핵심.** 1·2번은 로그를 읽으면 답이 나오지만, 3번은 6~7절의 내용으로 판단해야 함

- 채운 파일 **하나만** 올리면 제출이 끝남

막혀도 제출함

실행이 도중에 실패하면 오류 메시지가 그 파일에 자동으로 담기고, 질문이 "어디서 막혔나"로 바뀜
어디서 왜 막혔는지 적은 제출도 온전한 제출. 실행 성공 여부로 점수를 매기지 않음

이 장은 학번을 난수 시드로 씀

그래서 제출 파일의 숫자는 교재의 값과, 그리고 옆 사람의 값과 다른 것이 정상임

숫자는 달라도 **무작위 검증이 블록 검증보다 높다는 방향**은 같아야 함. 방향까지 다르면 그 이유를 찾는 것부터가 과제